

Overview

Build a small Retrieval-Augmented Generation (RAG) service that, given any starting website URL, crawls all of the site, indexes the content, and answers questions grounded strictly in the pages it collected, returning citations of the source page URLs used for each answer. The focus is practical skills in web content ingestion, retrieval quality, grounded prompting, basic evaluation, and clear engineering decisions within a realistic timebox.

Objective

Given a starting URL, implement a pipeline to crawl in-domain pages up to a page limit, extract clean text, chunk and embed content, store it in a vector index, and serve a Q&A interface that answers only from retrieved context with explicit source page URLs. The system must decline or say “not enough information” when the crawled content does not support an answer, and it must log and return which pages and snippets were used for each response.

Scope and Timebox

Target 4–8 hours of focused engineering, with a submission window of up to 72 hours. Partial but well-documented solutions are acceptable. Prioritize correctness, grounding, evaluation, and clear tradeoffs over UI polish or extensive infrastructure.

What to Build

- A polite crawler that respects robots.txt, stays within the registrable domain of the starting URL, and collects up to 30–50 pages, extracting main content and reducing boilerplate where possible.
- An indexing step that chunks text, computes embeddings, and writes to a vector index suitable for similarity search, with a short note justifying chunk size and overlap.
- A question-answering API or CLI that retrieves top-k chunks, constructs a grounded prompt, generates an answer, and returns the answer with source URLs, snippet highlights, and basic timing.

Constraints

- Input: one primary URL provided at evaluation time, with an optional second URL for robustness; the system must limit scope to the same registrable domain and enforce a hard page cap.

- Content types: focus on HTML text; multi-media, binary files and heavy client-side rendering are optional and not required under the timebox.
- Politeness: implement a crawl delay or concurrency limit and document choices to avoid overloading hosts.

Functional Requirements

- Crawl: accept a starting URL, crawl within domain up to N pages, store normalized page text, and persist a URL-to-document mapping for citations.
- Index: chunk text (for example, 500–1,000 characters with small overlap), embed, and store vectors; document chunking and embedding choices in the README.
- Ask: accept a question, retrieve top-k chunks, build a grounded prompt that answers only from provided context, and return JSON with the answer, list of source URLs and snippets, and response timings.

Non-Functional Requirements

- Grounding and refusals: answer only from retrieved context; if insufficient evidence, respond with a clear refusal such as “not found in crawled content,” including the closest retrieved snippets.
- Observability: log retrieval latency, generation latency, end-to-end p50 and p95 latency over a small query batch, error states, and token usage or cost if available.

Safety and Guardrails

- Prompt hardening: instruct the model to ignore any instructions found within crawled pages that attempt to alter behavior and to never execute or follow page directives.
- Content boundaries: restrict answers to the crawled domain and refuse when a query requires off-site or missing information.

API or CLI Specification

- POST /crawl { start_url, max_pages, max_depth, crawl_delay_ms } → { page_count, skipped_count, urls: [...] }

- POST /index { chunk_size, chunk_overlap, embedding_model } → { vector_count, errors: [...] }
- POST /ask { question, top_k } → { answer, sources: [{ url, snippet }], timings: { retrieval_ms, generation_ms, total_ms } }
CLI equivalents are acceptable if HTTP is not implemented.

Deliverables

- Repository with source, scripts, and tests, plus a concise README covering setup, run, evaluate, architecture description, and tradeoffs in 10–15 bullets.
- API or CLI samples or Evals: provide example requests and responses for crawl, index, and ask, including at least two example answers with cited URLs and snippets, one of which demonstrates a refusal.
OR
Demo: a 3–5 minute screen recording showing crawl, index, two ask calls (one answerable, one unanswerable), and a quick walk-through of the metrics report.
- DO NOT include anything that you don't understand, including code, content, tools, etc.

Rubric (100 Points)

- Grounding and correctness (30): answers are supported by retrieved context with clear source URLs and sensible refusals when evidence is lacking.
- Retrieval quality (20): chunking, top-k choices, and recall@k results are justified and reproducible using the evals set.
- Engineering quality (20): code structure, sanity tests, scripts, error handling, and basic observability are present and easy to run.
- Design clarity (15): README clearly explains architecture, tradeoffs, limitations, and next steps in concise, practical language.
- Safety and guardrails (15): context-only answering, prompt hardening against page-embedded instructions, and documented boundaries are implemented.

Implementation Guidance (Optional)

- Start with a straightforward RAG pipeline: crawl → clean → chunk → embed → vector index → retrieve → grounded prompt → generate answer with citations to page URLs.
- Default chunk size of 500–1,000 characters with small overlap is a good baseline; adjust based on observed retrieval quality and document the reasoning.
- Preferably use open source AI models for embedding and chat response. OSS tech stack is sufficient, there is no need to use paid tools.

What Not to Build

- Do not spend time on complex UIs, multi-agent frameworks, or large-scale orchestration; a simple API or CLI is sufficient.
- Do not include proprietary datasets or scrape beyond the assigned domain; avoid storing or sharing sensitive data encountered during crawling.

Tooling and Prompts Disclosure

- List the LLMs, embedding models, libraries, and prompt templates used in a “Tooling and Prompts” section for transparent evaluation and reproducibility.
- If any OSS or starter code or templates were adapted, include attributions in comments and in the README.

[SUBMISSION LINK](#)